[BUG] qml.transforms.transpile does not work with MultiRZ

https://github.com/PennyLaneAI/pennylane/issues/3100

ROW 78

[BUG] qml.transforms.transpile does not work with MultiRZ #3100

New issue

Closed #3104

QuantumFall opened on Sep 22, 2022 · edited by QuantumFall

Edits

Expected behavior

qml.transforms.transpile does not add SWAP gates for MultiRZ operations between 2 qubits and just ignores it. This might be due to MultiRZ being able to act on more than 2 qubits at a time. However, it's currently the only way to add RZZ gates between 2 qubits.

This makes it difficult to see the transpiled circuit for QAOA setups as attempting to transpile a circuit constructed from `qml.qaoa` functions just produces the decomposition of the problem unitary (aka U_H) into RZZ gates that don't match the coupling map.

Short example (expected behaviour using CX gates):

```
def nwise(lst, k=2):
    # Function to obtain [(0,1), (1,2), (2,3), ... ], i.e. linear coupling map
    return list(zip(*[lst[i:] for i in range(k)]))

dev = qml.device('default.qubit', wires=4)

@qml.qnode(dev)
@qml.transforms.transpile(coupling_map=nwise(np.arange(4)))
def circuit(param):
    qml.CNOT(wires=[0, 1])
    qml.CNOT(wires=[0, 2])
    qml.CNOT(wires=[0, 3])
    qml.PhaseShift(param, wires=0)
    return qml.probs(wires=[0, 1, 2, 3])
print(qml.draw(circuit)(0.6))

>>>

0: ─╭●─────╭●─────╭●─R𝜙(0.60)─┤ ╭Probs
1: ─╰X─╭SWAP─╰X─╭SWAP─╰X──────┤ ├Probs
2: ────╰SWAP─╭SWAP─╰SWAP──────┤ ├Probs
3: ──────────╰SWAP────────────┤ ╰Probs
```

Actual behavior

But changing to MultiRZ (which is currently the only way to implement RZZ unless I manually decomp them into CX-RZ-CX) gives:

```
def nwise(lst, k=2):
    # Function to obtain [(0,1), (1,2), (2,3), ... ], i.e. linear coupling map
    return list(zip(*[lst[i:] for i in range(k)]))

dev = qml.device('default.qubit', wires=4)

@qml.qnode(dev)
@qml.transforms.transpile(coupling_map=nwise(np.arange(4)))
def circuit(param):
    qml.MultiRZ(param, wires=[0, 1])
    qml.MultiRZ(param, wires=[0, 2])
    qml.MultiRZ(param, wires=[0, 3])
    qml.PhaseShift(param, wires=0)
    return qml.probs(wires=[0, 1, 2, 3])
print(qml.draw(circuit)(0.6))

0: ─╭MultiRZ(0.60)─╭MultiRZ(0.60)─╭MultiRZ(0.60)─R𝜙(0.60)─┤ ╭Probs
1: ─╰MultiRZ(0.60)─│──────────────│──────────────────────┤ ├Probs
2: ────────────────╰MultiRZ(0.60)─│──────────────────────┤ ├Probs
3: ──────────────────────────────╰MultiRZ(0.60)──────────┤ ╰Probs
```

Assignees
No one assigned

Labels
bug

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
Fix `qml.transforms.transpile` for ops with `AnyWires` acting on two-qubits
PennyLaneAI/pennylane

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

Additional information

Full QAOA example below:

Source code

```
import numpy as np
import networkx as nx

import pennylane as qml
from pennylane import numpy as plnp
from pennylane import qaoa


def rand_mc(nv, deg, seed):
    # Function to generate max-cut matrices of random n-variable d-regular graphs
    graph = nx.generators.random_graphs.random_regular_graph(deg, nv, seed=seed)
    matrix = nx.to_numpy_array(graph, nodelist=np.arange(nv))

    for k in range(nv):
        matrix[k][k] = -np.sum(matrix[k])
    return matrix, seed


def get_hvals(matrix):
    # For max cut problem, this is already written in a form to be minimized.
    # So the ground state of the resulting Hamiltonian will provide the solution to the max cut problem.

    # Gets the Ising coefficients, NOT the 2^n by 2^n Ising Hamiltonian.
    # Reparameterizes the QUBO problem into {+1, -1}
    # Reparameterization done using x=(Z+1)/2
    # Minimize z.J.z + z.h + offset
    nvars = len(matrix)
    jmat = np.zeros(shape=(nvars,nvars))
    hvec = np.zeros(nvars)

    for i in range(nvars):
        for j in range(nvars):
            if i == j:
                hvec[i] = np.sum(matrix[i])/2
            else:
                jmat[i][j] = matrix[i][j]/4
                jmat[j][i] = matrix[i][j]/4

    # Gives the correct offset value to the CF
    offset = (np.sum(matrix)/4 + np.trace(matrix)/4)
    return jmat, hvec, offset


def get_qml_ising(matrix):
    # Function to obtain qml.Hamiltonian from Ising coefficients
    nq = len(matrix)
    jj, hh, oo = get_hvals(matrix)
    h_coeff = []
    h_obs = []

    for i in range(nq):
        for j in range(i,nq):
            if i == j:
                h_coeff.append(hh[i])
                h_obs.append(qml.PauliZ(i))
            else:
                h_coeff.append(2*jj[i,j])
                h_obs.append( qml.PauliZ(i) @ qml.PauliZ(j))

    h_coeff.append(oo)
    h_coeff = np.array(h_coeff)
    h_obs.append(qml.Identity(0))
    hamiltonian = qml.Hamiltonian(h_coeff, h_obs)
    return hamiltonian

def get_qaoa_mixer(matrix):
    # Function to obtain QAOA mixer Hamiltonian
    nq = len(matrix)
    h_coeff = []
    h_obs = []

    for i in range(nq):
```

```python
    for i in range(nq):
        h_coeff.append(1)
        h_obs.append(qml.PauliX(i))
    hamiltonian = qml.Hamiltonian(h_coeff, h_obs)
    return hamiltonian


def qaoa_circ(params, wires = None, problem_hamiltonian = None, mixing_hamiltonian = None):
    # QAOA Circuit
    P = len(params)//2
    nq = len(problem_hamiltonian.wires)

    def qaoa_layer(gamma, beta):
        qaoa.cost_layer(gamma, problem_hamiltonian)
        qaoa.mixer_layer(beta, mixing_hamiltonian)

    # g1, b1, g2, b2, etc....
    gamma_list = params[::2][:P] # for U_H
    beta_list = params[1::2][:P] # for U_X

    for nq_ in range(nq): # apply Hadamards to get all in x-basis
        qml.Hadamard(wires=nq_)
    qml.layer(qaoa_layer, P, gamma_list, beta_list) # p instances of unitary operators

    # Returns probs because measurement of H not supported according to documents
    # https://docs.pennylane.ai/en/stable/code/api/pennylane.transforms.transpile.html
    return qml.probs(wires=np.arange(nq))


def nwise(lst, k=2):
    # Function to obtain [(0,1), (1,2), (2,3), ... ], i.e. linear coupling map
    return list(zip(*[lst[i:] for i in range(k)]))

#+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

nqq = 4                                                                    # Number of qubits
tmat = rand_mc(nqq, 3, seed=1029)[0]                                       # n-variable 3-regular graph
n_measurements = None                                                      # Use statevector
coupling_map = nwise(np.arange(nqq))                                       # Get coupling map

hamiltonian = get_qml_ising(tmat)                                          # Get Pennylane Hamiltonian
mixing_hamiltonian = get_qaoa_mixer(tmat)


dev = qml.device('default.qubit', wires=nqq)                               # Initialize device
transpiled_qaoa = qml.transforms.transpile(coupling_map=coupling_map)(qaoa_circ)
transpiled_qaoa_qnode = qml.QNode(transpiled_qaoa, dev)
print(qml.draw(transpiled_qaoa_qnode)([1,1], wires = None, problem_hamiltonian = hamiltonian, mixing_hamiltoni

# Output is supposed to be transpiled according to linear coupling map which includes a bunch of SWAP gates.
>>>
0: ──H──RZ(0.00)─╭RZZ(1.00)─╭RZZ(1.00)─╭RZZ(1.00)──RX(2.00)────────────────────────────────────
1: ──H───────────╰RZZ(1.00)─│──────────│──────────────RZ(0.00)─╭RZZ(1.00)─╭RZZ(1.00)──RX(2.00)
2: ──H──────────────────────╰RZZ(1.00)─│───────────────────────╰RZZ(1.00)─│───────────RZ(0.00)
3: ──H─────────────────────────────────╰RZZ(1.00)─────────────────────────╰RZZ(1.00)──────────

─────────────────────────────────┤ ╭Probs
─────────────────────────────────┤ ├Probs
──╭RZZ(1.00)──RX(2.00)───────────┤ ├Probs
──╰RZZ(1.00)──RZ(0.00)──RX(2.00)─┤ ╰Probs
```

Tracebacks

No response

System information

```
Name: PennyLane
Version: 0.26.0

Platform info:           Windows-10-10.0.19041-SP0
Python version:          3.7.13
Numpy version:           1.21.5
Scipy version:           1.7.3
Installed devices:
- default.gaussian (PennyLane-0.26.0)
- default.mixed (PennyLane-0.26.0)
- default.qubit (PennyLane-0.26.0)
- default.qubit.autograd (PennyLane-0.26.0)
```

```
- default.qubit.jax (PennyLane-0.26.0)
- default.qubit.tf (PennyLane-0.26.0)
- default.qubit.torch (PennyLane-0.26.0)
- default.qutrit (PennyLane-0.26.0)
- lightning.qubit (PennyLane-Lightning-0.26.0)
- cirq.mixedsimulator (PennyLane-Cirq-0.22.0)
- cirq.pasqal (PennyLane-Cirq-0.22.0)
- cirq.qsim (PennyLane-Cirq-0.22.0)
- cirq.qsimh (PennyLane-Cirq-0.22.0)
- cirq.simulator (PennyLane-Cirq-0.22.0)
- qiskit.aer (PennyLane-qiskit-0.21.0)
- qiskit.basicaer (PennyLane-qiskit-0.21.0)
- qiskit.ibmq (PennyLane-qiskit-0.21.0)
- qiskit.ibmq.circuit_runner (PennyLane-qiskit-0.21.0)
- qiskit.ibmq.sampler (PennyLane-qiskit-0.21.0)
- forest.numpy_wavefunction (PennyLane-Forest-0.21.0.dev0)
- forest.qvm (PennyLane-Forest-0.21.0.dev0)
- forest.wavefunction (PennyLane-Forest-0.21.0.dev0)
```

Existing GitHub issues

- [x] I have searched existing GitHub issues to make sure the issue does not already exist.

QuantumFall added `bug 🦠` on Sep 22, 2022

CatalinaAlbornoz on Sep 22, 2022 — Contributor

Hi @QuantumFall, thank you for reporting this bug! We will look into it.

antalszava mentioned this on Sep 23, 2022

Fix `qml.transforms.transpile` for ops with `AnyWires` acting on two-qubits #3104

antalszava closed this as completed in #3104 on Sep 30, 2022

antalszava on Sep 30, 2022 — Contributor

Hi @QuantumFall, the example you shared should work now on the `master` branch of the PennyLane repo. Support for gates acting on more than 2 qubits is future work with `transpile`.

Fix `qml.transforms.transpile` for ops with `AnyWires` acting on two-qubits #3104

Code

Merged antalszava merged 8 commits into `master` from `fix_transpile` on Sep 30, 2022

Conversation 7 | Commits 8 | Checks 0 | Files changed 3 | +131 −2

antalszava commented on Sep 23, 2022 • edited — Contributor

Context:
A quantum function with the `qml.MultiRZ` operation doesn't get transpiled correctly with the `qml.transforms.transpile` transform.

Description of the Change:

Turns out that the issue is that `qml.MultiRZ` acts on `AnyWires` and incorrect logic is being used in the `transpile` transform. Changes the condition that creates the issue.

Benefits:

`qml.MultiRZ` is compatible with `transpile` when it acts on two-qubits.

Possible Drawbacks:

Another finding: the `transpile` transform likely has to consider more than two-qubit gates too. The logic seems to have assumed that all gates are acting on either 1 or 2 qubits.
#3120
[sc-26639]

Related GitHub Issues:
Closes #3100

Reviewers
AlbertMitjans ✓
albi3ro •

Assignees
No one assigned

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
[BUG] qml.transforms.transpile does not wor...